

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap

- Vorstellungen Aufgaben
- Feedback Übungsprojekt
- Q&A Dokumente, um wieder in die Themen reinzukommen
 - OOP_Cpp_Part_6_Q&A

Kapitel 4: Die Standard Template Library



41. Container

42. Iteratoren

43. Algorithmen

44. Templates

jeder Baustein braucht den vorigen

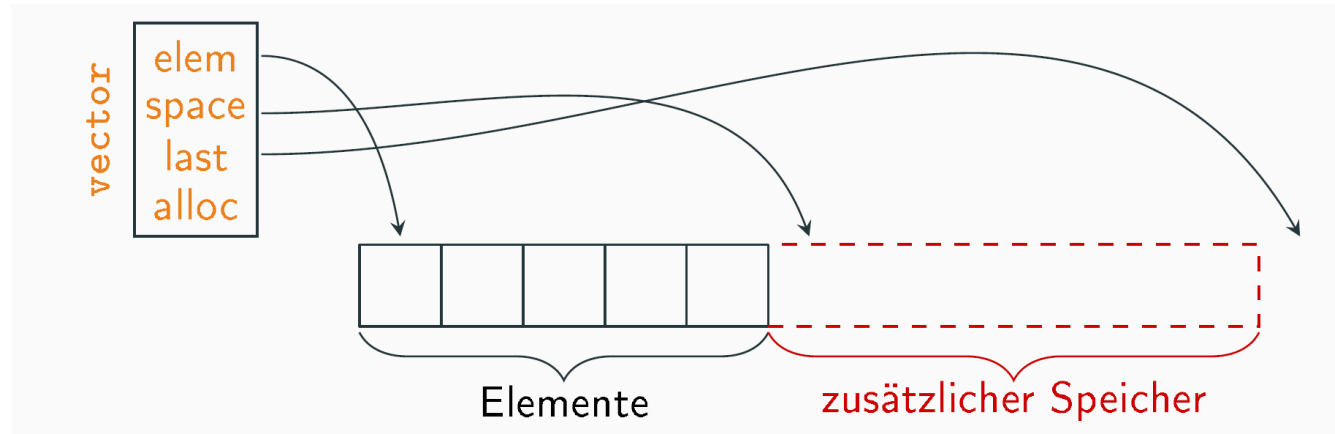
STL - Container

- Speichern eine Menge von Variablen strukturiert (immer nur ein Datentyp!)
- **Speichern Werte** und nicht Referenzen
- Stellen Methoden zur Verfügung:
`insert()`, `clear()`, `begin()`, `end()`, `size()`
- Container unterscheiden sich durch Funktionsumfang und Laufzeiten teilweise deutlich!
- Einen STL Container kennen Sie bereits: den `Vector`!
Dieser gilt als Standardcontainer.

<https://cplusplus.com/reference/stl/>

Container - `std::vector`

- `std::vector` ist eine Sequenz von **Elementen gleichen Typs** die auf dem **Heap** aufeinanderfolgend abgelegt werden
- Dazu werden im sogenannten *Handler* noch zusätzlich gespeichert:
 - ein Pointer auf das erste Element, einer hinter das letzte Element und hinter den gesamt allokierten Speicher
- Zudem wird ein *Allokator* gespeichert über den intern neuer Speicher angefordert werden kann



Container - `std::vector`

- Vector erstellt intern dynamisch ein Array
- **Nachteile:**
 - Bei jedem Insert/Remove innerhalb des Arrays muss ein neues Array allokiert und die Elemente umgezogen werden → Viel Rechenzeit!
 - Es wird schon von Vornherein mehr Speicher allokiert als direkt benötigt wird
- **Vorteile:**
 - Sehr effizienter Zugriff auf Elemente
 - **Warum? Da:**
 - **Effizient? Für:**

std::vector vs std::array

- Arrays haben fixe Größe
- Brauchen weniger Speicherplatz
- **Genauso effizient beim Zugriff auf einzelne Werte**
- Sollte anstelle eines `Vector`s benutzt werden, **wenn?**
- Initialisierung:
 - `std::array<class T, size n>`
 - **Hilfreicher als `T[n]`, da `T[n]` keine Convenience-Features und Iteratoren beinhaltet!**
 - Wird als einziger Container auf dem Stack initialisiert

Container – `std::deque`

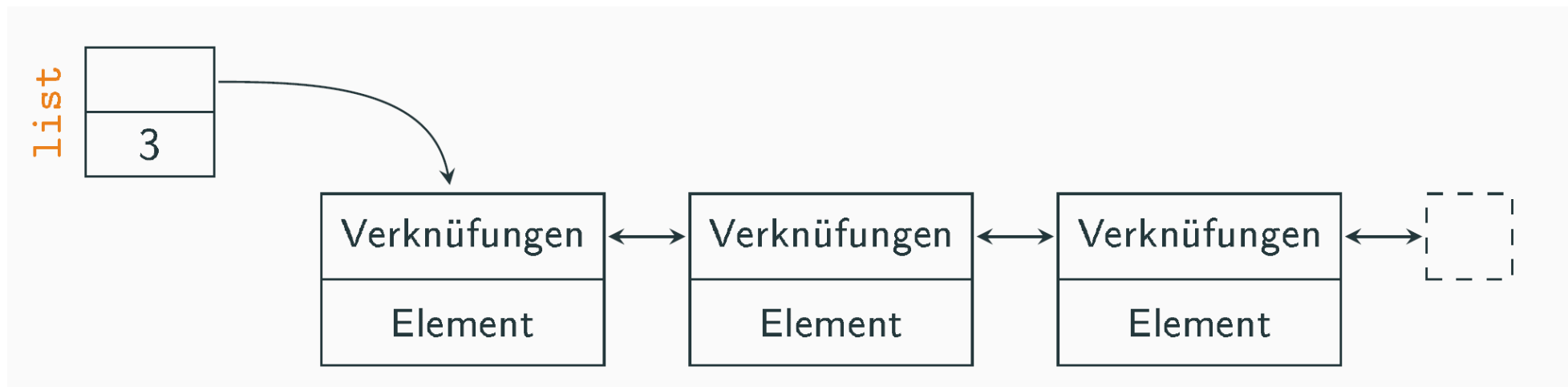
- `std::deque<class T, class Alloc = allocator<T>>`
- Double-Ended-Queue
- Etwas komplexer als `std::vector`
- **Es können sowohl am Ende als auch am Anfang effizient Elemente gelöscht oder hinzugefügt werden**

Container – `std::list`

- **Entspricht doppelt-verketteter Liste aus C**
- Jedes Element speichert eine Referenz auf seinen Vorgänger und seinen Nachfolger
- **Vorteile:**
 - Effizientes Hinzufügen und Löschen an beliebiger Position
- **Nachteile:**
 - Ineffizient beim Zugriff auf Element einer bestimmten Position: **Warum?**
 - Ineffizienz wächst mit Größe
- `std::forward_list`: Speichert nur eine Referenz auf Nachfolgeelement, verbraucht damit weniger Speicherplatz, kann aber auch nur in eine Richtung iterieren

Container – `std::list`

- List bietet *stable storage*: Hinzufügen und Entfernen von Elementen ohne Verschieben der anderen im Arbeitsspeicher
- Einbinden über `<list>` Header



Container – `std::stack`

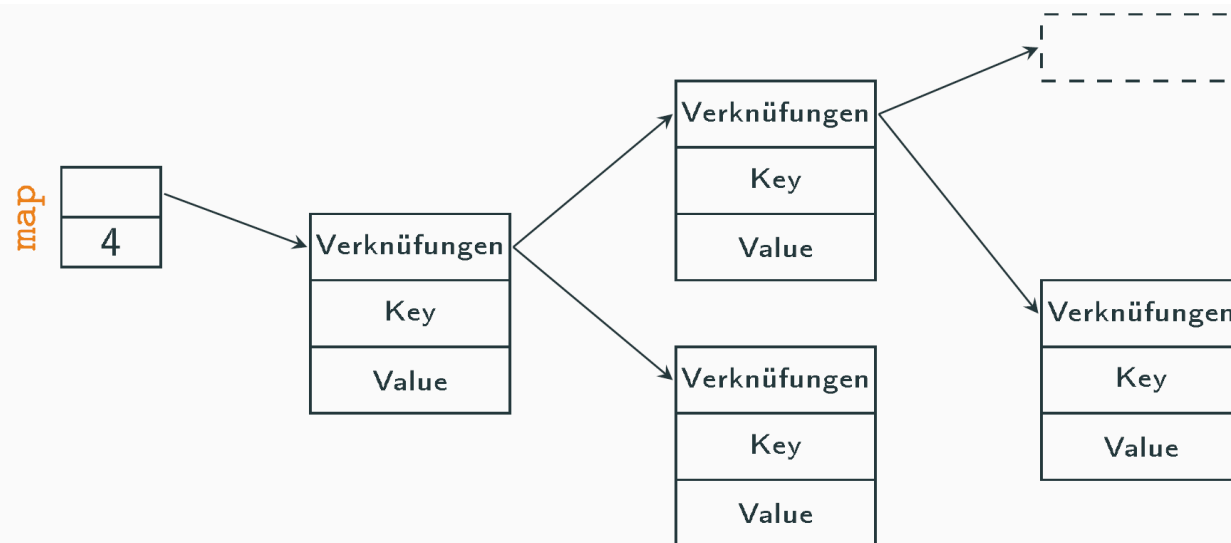
- `std::stack<class T, class Container = deque <T>>`
- Englisch: Stapel
- Ermöglicht nur `top`, `push` und `pop` Befehle: Es kann immer nur das „oberste“ also das als letztes zugefügte Element verändert werden
- `pop`: Entfernt das oberste Element
- `push`: Fügt ein neues oberstes Element hinzu
- `top`: Gibt das oberste Element aus ohne es zu verändern
- Last-in-First-out: **LIFO**
- Verwendung z.B. in Betriebssystemen: Callstack

Container – std::queue

- `std::queue<class T, class Container = deque<T>>`
- Englisch: Schlange
- First-in-first-out: **FIFO**
- Liefert Funktionen zum hinten anfügen und vorne wegnehmen
- Priorität kann auch überschrieben werden: nicht immer einfach das Element raus nehmen was am längsten drin war

Container – std::map (assoziative Container)

- Key-Value-Paare: Häufig ein einfacher Datentyp der auf einen komplexeren Datentyp hinweist
- Schneller Zugriff auf Wert, bei gegebenem Key
- Neben `std::map` gibt es noch viele Abwandlungen: Hash, Set, Hashmap, ...
- Es gibt immer eine `ordered` (standard) und eine `unordered_` Version. Letztere ist schneller im Zugriff, dafür nicht iterierbar



```
map<char,int> first;
```

```
//initializing
first['a']=10;
first['b']=20;
first['c']=30;
first['d']=40;
```

Container Quiz

- Was für einen Container (und warum) nutze ich in einer Supermarktverwaltung, in der ich jedes Produkt durch seine Produktnummer identifizieren kann?
- Was für einen Container nutze ich für eine Liste mit Fehlermeldungen, die ich während einer Berechnung sammle und dem Nutzer am Ende zeige?
- Was für einen Container (und warum) nutzt man für die Implementierung der Türme von Hanoi?

Weitere Assoziative Container

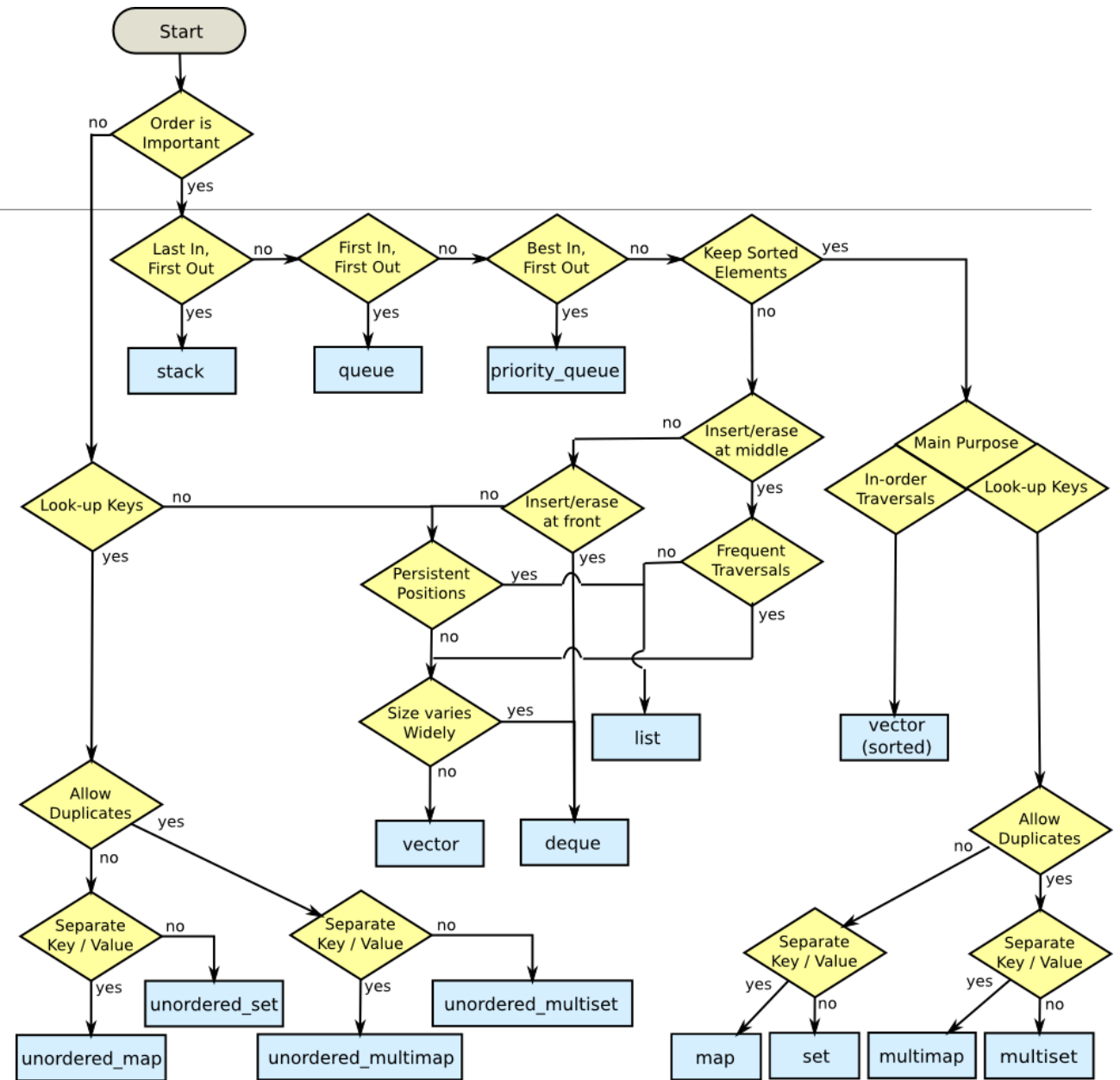
- Eigenschaften
 - Speichern Elemente in einer bestimmten Reihenfolge
 - Basieren auf Schlüsseln
 - Gut für schnelle Suchen, Einfügungen und Löschungen.
- `Std::set` – Sammlung von eindeutigen Schlüsseln, automatisch sortiert
- *Std::multiset* – Ähnlich set, aber mit doppelten Schlüsseln erlaubt
- *Std::multimap* – Ähnlich wie map, aber mit doppelten Schlüsseln erlaubt

Weitere Unordered Container

- Eigenschaften
 - Hash-Tabellen-basierte Container
 - Gut für schnellen Zugriff auf Elemente basierend auf Hashing
 - Gut geeignet für schnelles suchen
- `std::unordered_set` – Pendant zu `std::set`
- `std::unordered_multiset` – Pendant zu `std::multiset`
- `std::unordered_map` - Pendant zu `std::map`
- `std::unordered_multimap` – Pendant zu `std::multimap`

Welcher Container?

- Hilfestellung um den richtigen Container zu finden



Container Quiz Part 2

- Sie entwickeln eine Anwendung zur Verwaltung einer Aufgabenliste, bei der Benutzer häufig neue Aufgaben hinzufügen und erledigte Aufgaben entfernen. Die Benutzer wollen zudem jederzeit schnell auf jede Aufgabe in der Liste zugreifen können. Welcher Container ist hierfür am besten geeignet?
- Sie schreiben ein Programm, das große Mengen an Studentendaten speichert, bei denen die Studenten über ihre Matrikelnummer identifiziert werden. Sie müssen häufig nach bestimmten Matrikelnummern suchen, neue Studenten hinzufügen und bestehende Studenten entfernen, wobei die Reihenfolge der Einträge keine Rolle spielt. Welcher Container erfüllt diese Anforderungen am besten?
- In Ihrem Projekt arbeiten Sie an einer Anwendung, die eine Playlist verwaltet. Benutzer sollen in der Lage sein, Songs an beliebigen Positionen in der Playlist hinzuzufügen oder zu entfernen. Außerdem soll die Reihenfolge der Songs unverändert bleiben. Welcher Container eignet sich hierfür am besten?

Aufgaben: Container

- 1) Implementieren Sie ein Supermarkt Kassensystem
 - 1) Nutzen sie dazu `std::map` oder `std::unordered_map`
 - 2) Befüllen Sie den Container jeweils mit einem Produktnamen und einem Preis
 - 3) Über die Kommandozeile gibt ein Kunde seine Einkaufsliste ein
 - 4) Die eingegebenen Produktnamen sollen in einem Vector gespeichert werden
 - 5) Mithilfe des Vectors und der Map soll ein Kassenzettel Ausdruck in der Kommandozeile ausgegeben werden



Kapitel 4: Die Standard Template Library



41. Container

42. Iteratoren

43. Algorithmen

44. Templates

jeder Baustein braucht den vorigen

STL - Überblick



Iteratoren

- Ein Iterator ist ein Objekt, das auf ein Element in einer Folge von Elementen (z.B. `std::vector`, `std::array`) zeigt
- **Elementarste Form eines Iterators ist ein Pointer**
- **Die meisten STL-Container stellen Iteratoren zur Verfügung**
- Iteratoren ermöglichen generische Algorithmen die auf vielen Containern funktionieren wie z.B. Sortieren

Iteratoren

Iteratoren

```
01_lecture > 4_STL > 2_iteratoren > C++ main.cpp > main()
1  #include <vector>
2  #include <string>
3  #include <iostream>
4  #include <algorithm>
5
6  void printHelper(std::string str);
7
8  int main()
9  {
10
11     std::vector<std::string> myContainer;
12     myContainer = {"Hello", ",", " ", "World", "!"};
13
14     myContainer.push_back(" - range-based-for loop");
15     // range-based-for loop
16     for (std::string &str : myContainer)
17     {
18         std::cout << str;
19     }
20     std::cout << std::endl;
21     myContainer.pop_back();
22
23     myContainer.push_back(" - iterator-based loop");
24     // iterator-based loop
25     for (auto it = myContainer.begin(); it < myContainer.end(); it++)
26     {
27         // Iterator must be treated like a pointer
28         std::cout << *it;
29     }
30     std::cout << std::endl;
31     myContainer.pop_back();
32
33     myContainer.push_back(" - for_each loops");
34     // for_each loops
35     std::for_each(myContainer.begin(), myContainer.end(), printHelper);
36     std::cout << std::endl;
37
38     return 0;
39 }
40
```


Kapitel 4: Die Standard Template Library

41. Container

42. Iteratoren



43. Algorithmen

44. Templates

jeder Baustein braucht den vorigen

Überblick über C++ STL

Talk by Jonathan Boccara Cppcon 2018

Map [The World Map of C++ STL Algorithms - Fluent C++ \(fluentcpp.com\)](http://fluentcpp.com)



Warum STL-Algorithmen?

- Effizienz und Optimierung
- Fehlervermeidung
- Lesbarkeit und Wartbarkeit
- Konsistenz
- Zeiteffizienz
- Portabilität

Container sortieren

- `#include <algorithm>`
enthält neben `std::for_each`
auch viele andere Algorithmen wie
z.B. `std::sort`
- Als Default wird immer der `<`
Operator verwendet
- Kann aber überschrieben werden

Container sortieren

- `#include <algorithm>`
enthält neben `std::for_each`
auch viele andere Algorithmen wie
z.B. `std::sort`
- Als Default wird immer der `<`
Operator verwendet
- Kann aber überschrieben werden

```
01_lecture > 4_STL > 3_sort > C++ main.cpp > printContainer(std::vector<std::string>)\n1  #include <vector>\n2  #include <string>\n3  #include <iostream>\n4  #include <algorithm>\n5\n6  bool myWayToSort(std::string strA, std::string strB)\n7  {\n8      return (strA.length() > strB.length());\n9  }\n10\n11 void printContainer(std::vector<std::string> container)\n12 {\n13     // range-based-for loop\n14     for (std::string &str : container)\n15     {\n16         std::cout << str << " "; \n17     }\n18     std::cout << std::endl;\n19 }\n20\n21 int main()\n22 {\n23\n24     std::vector<std::string> myContainer;\n25     myContainer = {"Hello", ",", "World", "!", "I", "love", "std::sort", ".", "You", "too", "?"};\n26     printContainer(myContainer);\n27     std::cout << std::endl;\n28\n29     std::sort(myContainer.begin(), myContainer.end(), myWayToSort);\n30     printContainer(myContainer);\n31     std::cout << std::endl;\n32\n33     return 0;\n34 }
```

Aufgaben I: STL

- Erstellen Sie einen Vektor.
- Fügen Sie die Strings „Eier“, „Schmalz“, „Zucker“, „Salz“ und „Milch“ und „Mehl“ hinzu.
- Geben Sie ihn aus.
- Das dritte Element entfernen. Wieder ausgeben.
- „Schokolade“ hinzufügen, ausgeben.
- Das Letzte Element mit „Safran“ ersetzen, ausgeben.
- Nach „Schmalz“ suchen und es entferne. Wieder ausgeben.
- Sortieren Sie ihren Vector. Wieder ausgeben.

Aufgaben II: STL

- Auch hier: nach jedem Schritt ausgeben.
- Erstellen Sie eine Liste.
- Fügen Sie die Integer 0, 1, 1, 14, 5, 2, 3 hinzu.
- Entfernen Sie das erste Element.
- Fügen Sie die 8 hinzu.
- Ersetzen Sie die 14 mit einer 13 (ohne zu wissen an welcher Stelle die 14 steht).
- Sortieren Sie die Liste aufsteigend.
- Sortieren Sie die Liste absteigend.
- Sortieren Sie die Liste nach geraden und ungeraden Zahlen.

Kapitel 4: Die Standard Template Library

41. Container

42. Iteratoren

43. Algorithmen



44. Templates

jeder Baustein braucht den vorigen

Motivation: Wieder verwendbarer Code!

Motivation: Wieder verwendbarer Code!

```
01_lecture > 4_STL > 4_templates > C++ main.cpp > main()
1  #include <vector>
2  #include <list>
3  #include <iostream>
4  #include <string>
5  #include <algorithm>
6
7  > void printVector(const std::vector<std::string> &vec) ...
15
16 > void printList(const std::list<std::string> &inList) ...
24
25 template <class T>
26 void printContainer(const T &container)
27 {
28     for (const std::string &item : container)
29     {
30         std::cout << item << " ";
31     }
32     std::cout << std::endl;
33 }
34
35 int main()
36 {
37     std::vector<std::string> myVec = {"Eier", "std::vector", "Zucker", "Salz", "Milch", "Mehl"};
38
39     // specific functioncall
40     // printVector(myVec);
41     // printList(myVec); this is not possible (even it is nearly the same)
42
43     // compiler deduces type
44
45     // explicit type is given
46     printContainer<std::vector<std::string>>(myVec);
47
48     std::list<std::string> myList = {"Eier", "std::list", "Zucker", "Salz", "Milch", "Mehl"};
49     printContainer(myList);
50
51     return 0;
52 }
```

Templates – Anwendung

- Wiederverwenden von Funktionen mit gleichem Verhalten
DIE ABER **auf verschiedenen Datentypen arbeiten müssen**
- Funktionen in Bibliotheken bei denen unbekannt ist mit welchem Datentyp sie genutzt werden → **Dies ermöglicht erst die STL!**
- Funktion kriegt Decorator-keyword: `template`
- `T` wird standardmäßig als Platzhalter für die Klasse genommen
- Für mehrere Template-Parameter:
`template <class T, class U>`
- `T` kann auch als Return Type fungieren

```
template <class T>
void printContainer(T container)
{
    for (string& item : container)
    {
        cout << item << " ";
    }
    cout << endl;
}
```

Templates – Geht auch für Klassen

- Innerhalb von Klassen können auch Templates genutzt werden (gleiche Syntax)
- Compiler erstellt jeweils eine eigene Version der Funktion/der Klasse:
ROM steigt (Größe der .exe)!
- Templates haben Sie schon die ganze Zeit in Containern benutzt `vector<string>`
`myVector;`
- Intern: `vector::push_back(T value);`
- Welche Template Version aufgerufen werden soll kann beim Aufruf spezifiziert werden, z.B.: `printContainer<std::vector<int>>`
- Wenn es eindeutig aus dem Argument der Funktion/der Klasse ermittelt werden kann, muss nichts angegeben werden
- Anstelle von `class` kann auch `typename` verwendet werden (äquivalent)

Motivation: Wieder verwendbarer Code!

Motivation: Wieder verwendbarer Code!

```
//Template-Funktion zum Printen von Containern
template <class T>
void printContainer(const T& container)
{
    for (const string& item : container)
    {
        cout << item << " ";
    }
    cout << endl;
}

// Template-Klasse zum Printen von Containern
template <class T, char DIVIDER>
class ContainerPrinter
{
public:
    void print(const T& container) const
    {
        for (const string& item : container)
        {
            cout << item << DIVIDER;
        }
        cout << endl;
    }
};
```

```
int main()
{
    vector<string> myVec = {"Eier", "Schmalz", "Zucker", "Salz", "Milch", "Mehl"};
    // compiler deduces type
    printContainer(myVec);
    // explicit type is given
    printContainer<vector<string>>(myVec);

    list<string> myList = {"Eier", "Schmalz", "Zucker", "Salz", "Milch", "Mehl"};
    printContainer(myList);

    // Nutzung der Template-Klasse
    ContainerPrinter<vector<string>, ' '> vecPrinter;
    vecPrinter.print(myVec);
    ContainerPrinter<list<string>, ','> listPrinter;
    listPrinter.print(myList);

    return 0;
}
```

Aufgaben I: Templates

- 1) Implementieren Sie eine Funktion `minimumInt`, welche den Kleineren von zwei gegebenen Input Integer zurück gibt.
- 2) Schreiben Sie nun eine template Funktion `myMinimum`, welche das Kleinere von zwei gegebenen Input Werten zurück gibt wobei der Typ über einen Template Parameter angegeben wird (`int`, `double`, `float`, ...).
- 3) Implementieren Sie nun eine template Klasse `Pair`,
 - a) welche als zwei private Member zwei Instanzen einer beliebigen Klasse hat (`m_first` und `m_second`).
 - b) Es soll eine `public` Funktion besitzen `setPair(T a, T b)`, mit der beide privaten Member beschrieben werden können.
 - c) Dazu soll eine Funktion angeboten werden mit der das Minimum des gespeicherten Paares ausgegeben wird.

Aufgaben II: Übungsprojekt

1) Erweitern Sie ihr 4-Gewinnt Projekt:

- Rahmenbedingungen (z.B. maximale Füllhöhe des Feldes) und Gewinnbedingungen sollen nun abgefragt werden.

